So far, you have passed a simple array of strings into the `SelectList`. The `SelectList` assigned each element of the array to an option's text, resulting in the following HTML for the select element.

Listing 6.9 The generated HTML for the select city element

```
<select class="form-control" id="SelectedCity" name="SelectedCity">
    <option>London</option>
    <option>Berlin</option>
    <option>Paris</option>
    <option>Rome</option>
    <option>New York</option>
</select>
```

When an HTML option element only has text assigned to it, the text is passed as the value when the form is submitted. When you are working with data from a database (as you will, starting in chapter 8), you will more often want to work with the unique identifier associated with a data item in the database—its *primary key* value. These key values don't mean anything to the average user, so you will assign the keys to the `value` attribute of the select options while still passing in a piece of text that enables a user to understand what each option represents. An overload of the `SelectList` constructor enables you to map different values to the `value` attribute and the text of an option. The next example shows how to use this overload to mimic the process of mapping a numeric identifier for each city to the option value, while still presenting the name of the city to the user.

You will generate a collection of `City` objects, just like you did in the check box example in the previous chapter. You will assign these to the `SelectList` and specify which property represents the `DataValueField` (the option's value) and which one should be assigned to the `DataTextField` (the option's text). Because you are using a numeric key for each city, the `SelectedCity` property is